Async SQLAlchemy (SQLAlchemy 2.0 + FastAPI)

The main difference is:

```
# Sync
db=SessionLocal()
users=db.query(User).all()
```

becomes

```
# Async
result=await session.execute(select(User))
users=result.scalars().all()
```

SQLAlchemy provides:

- `create_async_engine()`
- `AsyncSession`
- `async_sessionmaker()`

Why Async?

Suppose 100 users hit your API simultaneously.

Sync

```
@app.get("/users")
defget_users():
users=db.query(User).all()
returnusers
```

Request 2 waits for Request 1.

Async

```
@app.get("/users")
asyncdefget_users():
    ...
```

While one request waits for the database, FastAPI can process other requests. This improves concurrency for I/O-bound workloads.

Step 1: Install Dependencies

For SQLite:

```
pip install sqlalchemy aiosqlite
```

For PostgreSQL:

```
pip install sqlalchemy asyncpg
```

SQLAlchemy's async layer requires an async-compatible database driver.

Step 2: Create Async Engine

Sync

```
engine=create_engine(
"sqlite:///users.db"
)
```

Async

```
fromsqlalchemy.ext.asyncioimportcreate_async_engine

engine=create_async_engine(
"sqlite+aiosqlite:///users.db",
echo=True
)

the echo=True parameter tells SQLAlchemy to print all SQL statements that it executes
to the console/logs.
```

Notice:

```
sqlite+aiosqlite:///
```

instead of:

```
sqlite:///
```

Step 3: Create Async Session

Sync

```
SessionLocal=sessionmaker(bind=engine)
```

Async

```
from sqlalchemy.ext.asyncioimport (AsyncSession,async_sessionmaker)

Async SessionLocal=async_sessionmaker(engine,class_=AsyncSession,expire_on_commit=Fals
e
)
```

`expire_on_commit=False` is commonly used so objects remain accessible after commit.

Step 4: Models Remain the Same

```
from  sqlalchemy.orm import declarative_base
from sqlalchemy import Column,Integer,String

Base=declarative_base()

class User(Base):

__tablename__="users"
```

```
id=Column(Integer,primary_key=True)
name=Column(String)
```

No async changes are needed in models.

Step 5: Create Tables

Sync

```
Base.metadata.create_all(bind=engine)
```

Async

```
@app.on_event("startup")
async def startup():
  async with engine.begin()as conn:
    await conn.run_sync(Base.metadata.create_all)
```

---

1. `@app.on_event("startup")`

This is a FastAPI decorator.

```
@app.on_event("startup")
```

It tells FastAPI:

> "Run this function once when the application starts."

Example

When you run:

```
uvicorn main:app--reload
```

FastAPI starts and automatically executes:

```
startup()
```

before handling any requests.

Think:

```
Start FastAPI
     ↓
Run startup()
     ↓
Application ready
     ↓
Accept requests
```

---

2. `async def startup():`

```
asyncdefstartup():
```

This creates an asynchronous function.

Because SQLAlchemy Async uses async operations, the function must be async.

Async function:

```
async def startup():
pass
```

Async functions allow:

```
await something()
```

---

3. `async with engine.begin() as conn:`

This is the most important line.

```
async with engine.begin() as conn:
```

What is `engine`?

Earlier:

```
engine=create_async_engine(
"sqlite+aiosqlite:///users.db"
)
```

The engine manages database connections.

What does `begin()` do?

It opens:

- a database connection
- a transaction

Equivalent idea:

```
conn=engine.connect()

# do work

conn.close()
```

But SQLAlchemy handles everything automatically.

Why `async with`?

Normal:

```
withopen("file.txt")asf:
pass
```

Async:

```
asyncwithengine.begin()asconn:
```

```
pass
```

When the block ends:

- transaction is committed
- connection is released

automatically.

Think:

```
Open connection
    ↓
Do database work
    ↓
Commit
    ↓
Close connection
```

---

4. `await conn.run_sync(...)`

```
await conn.run_sync(
Base.metadata.create_all
)
```

This confuses almost everyone initially.

Problem

`create_all()` is synchronous.

```
Base.metadata.create_all()
```

is NOT async.

But:

```
conn
```

is an async connection.

You cannot directly do:

```
await Base.metadata.create_all()
```

because `create_all()` is not a coroutine.

Solution

SQLAlchemy provides:

```
conn.run_sync(...)
```

which allows running synchronous code using an async connection.

Internally

You write:

```
await conn.run_sync(
Base.metadata.create_all
)
```

SQLAlchemy does roughly:

```
Base.metadata.create_all(sync_connection)
```

behind the scenes.

---

5. `Base.metadata.create_all`

What is this?

Suppose:

```
classUser(Base):
__tablename__="users"
```

```
classDepartment(Base):
__tablename__="departments"
```

All table definitions are stored in:

```
Base.metadata
```

When SQLAlchemy executes:

```
Base.metadata.create_all(...)
```

it generates SQL like:

```
CREATETABLE users (
    idINTEGERPRIMARYKEY,
    nameVARCHAR
);
```

```
CREATETABLE departments (
    idINTEGERPRIMARYKEY,
    nameVARCHAR
);
```

only if they don't already exist.

```
uvicorn main:app --reload
           ↓
FastAPI starts
           ↓
startup() executes
           ↓
engine.begin()
           ↓
Database connection opened
           ↓
create_all()
           ↓
Tables created
           ↓
Connection closed
```

```
          ↓
Server ready
```

Interview Answer

If asked:

> Why do we use `await conn.run_sync(Base.metadata.create_all)`?

Answer:

> `Base.metadata.create_all()` is a synchronous SQLAlchemy method. In Async SQLAlchemy we use `conn.run_sync()` to execute synchronous metadata operations through an async database connection. This allows tables to be created while still using the async engine and event loop.

```python
@app.get("/users",response_model=list[UserResponse])
async def get_users(db:AsyncSession=Depends(get_db)):

        result= await db.execute(select(User))

        users=result.scalars().all()

        return users
```

db: AsyncSession = Depends(get_db)

while writing the apis : `AsyncSession` is a SQLAlchemy class that represents a database session. A session is your interface for interacting with the database—it lets you add, update, delete, and query records.

`Depends(get_db)`

This is where FastAPI's dependency injection comes in.

```
Depends(get_db)
```

means:

> "Before calling this API, execute `get_db()`

create a get_db() fucntion in main.py

```
#create get_db(), which FastAPI will use as a dependency to
provide a database session for each request.
async def get_db():

    async with AsyncSessionLocal() as session:
        yield session
```

Think of `AsyncSessionLocal` as a **session factory**.

```
Engine
   |
   ▼
AsyncSessionLocal
   |
   ▼
Creates AsyncSession objects
```

Every time you write:

```
session=AsyncSessionLocal()
```

you get a **new database session**.

Why `yield` instead of `return`?

This is one of the most common FastAPI interview questions.

You might think:

```
async def get_db():
return AsyncSessionLocal()
```

But that has a problem.

If you `return`, FastAPI receives the session but has no built-in way to continue executing the function afterward to perform cleanup.

Instead, using:

```
yieldsession
```

pauses the function, gives the session to the endpoint, and then resumes the function after the endpoint finishes so cleanup can occur. This is why FastAPI recommends `yield` for database dependencies.

result = await db.execute(select(User))

users = result.scalars().all()

`select()` creates a SQL **SELECT statement**. It does **not** execute anything yet—it just builds the query. SQLAlchemy 2.0 recommends using `select()`

`db.execute(...)`

```
result=await db.execute(select(User))
```

Here:

- `db` is your `AsyncSession`.
- `execute()` sends the SQL query to the database.
- `await` is required because database communication is asynchronous.

`result.scalars()`

```
result.scalars()
```

removes the surrounding `Row` objects and gives you only the first element from each row.

Before `scalars()`:

```
Row(User(...))
Row(User(...))
```

After `scalars()`:

```
User(...)
User(...)
```

This is why `scalars()` is commonly used when selecting a single ORM entity such as `User`.

Step 4: `.all()`

```
users=result.scalars().all()
```

`all()` collects every object into a Python list.

So you finally get:

```

```

Sync vs Async Cheat Sheet

Sync	Async
`create_engine()`	`create_async_engine()`
`Session`	`AsyncSession`
`sessionmaker()`	`async_sessionmaker()`
`db.commit()`	`await db.commit()`
`db.refresh()`	`await db.refresh()`
`db.query(User)`	`select(User)`
`def endpoint()`	`async def endpoint()`

Sync	Async
`sqlite:///db.db`	`sqlite+aiosqlite:///db.db`